POLITECNICO
MILANO 1863

Field Robot Event 2026
Task 2

Simone Edmondo Bedini, Enrico Castelli, Mirko Usuelli, Leonardo Gargani

1 Introduction

This report aims to provide a comprehensive analysis of the work conducted for the Field Robot Event 2026 competition, specifically for Task 2. The task focuses on plant health detection in a maize field, requiring the robot to autonomously navigate through multiple curved rows within a three-minute time limit.

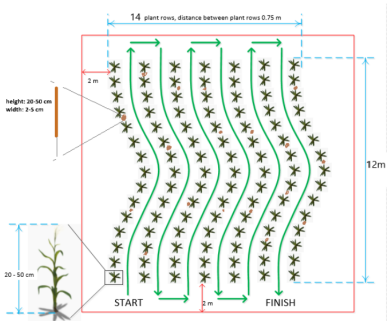


Figure 1: Task 2 curved maize field layout.

The primary objective is to identify signs of unhealthy plants, which are simulated using rectangular markers, made of jute or yellow paper, placed next to the plant. To effectively demonstrate this capability to the audience, the robot must trigger a clear public signal whenever a diseased plant is detected within a 40 cm detection area. Additionally, the robot is required to generate a digital log in CSV format detailing the row number and the precise distance of each identified plant from the start of the row.

The robot deployed for this challenge, named Grasslammer, is a four-wheeled autonomous platform equipped with multiple sensing and communication systems. These include two SICK single-plane LiDAR sensors, positioned at the front and rear of the robot, two OAK-D cameras installed on the lateral sides, and a downward-facing ZED stereo camera.

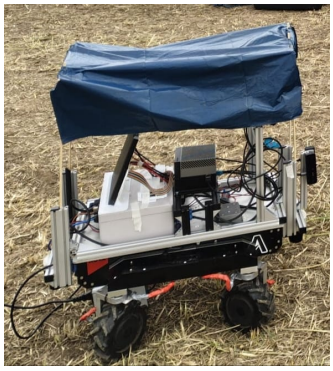


Figure 2: Grasslammer during field tests.

2 Problem Analysis

The objective of Task 2 can be decomposed into three primary sub-problems: visual *perception* of the markers, *localization and mapping* along curved trajectories, and real-time *visual feedback*.

The perception problem's main challenges are to ensure a consistent and reliable detection system in an external agricultural environment, where lighting is variable and visibility of the markers might be obstructed. The adopted solution involves using colour detection in the HSV colour space, alongside geometric and depth validation.

The second sub-problem involves mapping these

detections. The task rules mandate that the robot should record the row number and the distance of each plant from the start of the row, with a precision of 10 cm. Since relying solely on wheel odometry would lead to significant cumulative drift due to wheel slip and uneven soil, the solution is formulated as a semantic Graph SLAM problem, where continuous odometry updates and relative 3D marker detections are fused to jointly optimize the robot's trajectory and the landmark positions.

The third sub-problem addresses real-time visual feedback. To signal that the detection of a marker took place, a LED matrix is placed on the back of the robot, which displays an arrow pointing in the direction (left/right) of the marker. The visual signal has to be displayed while the marker is aligned with a 40 cm area marked on the robot, therefore the system needs to be sufficiently responsive to ensure that the whole detection pipeline triggers before the marker leaves the designated area.

In the following sections, we present in detail the approaches and techniques we used to overcome the aforementioned problems.

3 Perception

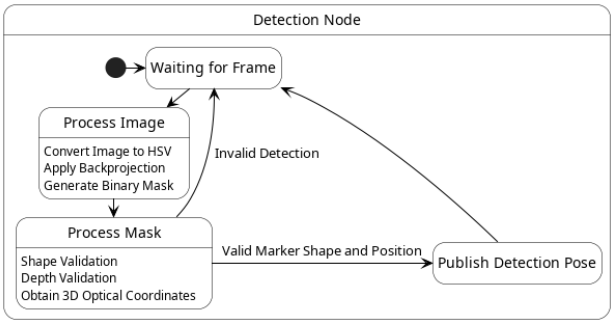


Figure 3: Diagram of the detection node pipeline.

3.1 HSV Histogram Backprojection

The core of the visual perception pipeline is executed by a colour detection node. To begin, the incoming BGR images are converted into the HSV (Hue, Saturation, Value) colour space, which separates chromaticity from intensity: this provides greater robustness against the shadows and variable lighting conditions typical of outdoor agricultural environments.

To isolate the markers in the image, the node employs histogram backprojection. Before a run, a calibration phase occurs where a frequency matrix of the Hue and Saturation values of each type of marker is computed: this ensures that the detector has values consistent with the current lighting conditions. The calibration is performed by adjusting a rectangular region of interest, through the use of ROS2's dynamic parameters, over a marker placed in front of the camera, and triggering a service call once the region overlaps the marker, which generates the histogram. The backprojection algorithm then evaluates the Hue and Saturation channels of the current frame against these histograms, generating a grayscale probability map where the intensity of each pixel represents the likelihood of it belonging to the target colour distribution.

This continuous probability map is then converted into a binary mask by applying a configurable probability threshold. To remove noise and refine the shape of the blob, a sequence of morphological operations is applied: an opening operation (*erosion*) removes isolated noisy pixels, while a closing operation (*dilation*) fills small gaps and solidifies the core structure of valid marker blobs. The resulting sanitized binary mask isolates the contiguous regions of the target colour.



Figure 4: Example of generated detection mask.

3.2 Geometric and Depth Validation

While HSV colour filtering is effective, it is susceptible to false positives from similarly coloured background elements and markers in different rows from the one traversed. To ensure that only valid markers are processed, a geometric and depth validation pipeline is applied to the extracted binary mask.

First, the algorithm extracts discrete contours from the mask and calculates their area, immedi-

ately discarding any contour that falls below a minimum area threshold. For the remaining candidates, the shape is then evaluated using two key geometric metrics: *aspect ratio* (the ratio of the longest side to the shortest side) and *extent* (the ratio of the contour's actual area to the area of its bounding box). By comparing these values against configurable thresholds, the system can reliably distinguish the rectangular target markers from irregular blobs.

Once a contour passes the 2D geometric checks, the system extracts its 3D position using the aligned depth image provided by the OAK-D camera. Invalid measurements, such as NaNs, zeroes, or values exceeding the maximum expected depth, are filtered out, and then the median depth value of the pixels belonging to the mask is computed.

Finally, using the camera's intrinsic parameters (focal length and principal point), the 2D center coordinates of the marker and its median depth are projected into 3D space within the camera's optical frame. If the marker falls within the required 40 cm lateral detection zone, the 3D point is transformed into the robot's central reference frame, to be subsequently handled by the mapping and localization algorithm.

4 Mapping and Localization

To determine the location of the detected markers in each row, and combat the inevitable odometry drift caused by traversing the rough terrain in a field, the system relies on a semantic Graph SLAM architecture [1]. This is implemented through the `orchards-semantic-slam` library, as a wrapper for the GTSAM optimization engine [2].

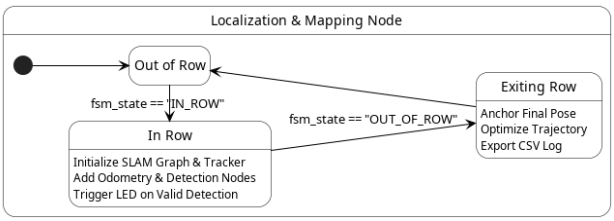


Figure 5: Diagram of the mapper node pipeline.

4.1 Multi-Object Tracking

Before raw visual detections are integrated into the SLAM graph, they must be filtered for temporal consistency to prevent multiple detections of the same marker from being recognized as separate objects, or spurious reflections from corrupting the map. This is managed by the `Tracker` module from the `orchards-semantic-slam` library, which handles the spatial and semantic data association of the measurements across successive frames [2].

When the system processes a new marker detection, the library attempts to match it to existing tracks using an optimal assignment strategy. The tracker first computes a comprehensive cost matrix representing the distance between all existing tracks and new detections. The cost is composed of a spatial component relying on the Euclidean distance, where any distance exceeding a configurable gating threshold is artificially penalized with a massive cost to prevent physically impossible associations, and a semantic component, which similarly applies a massive penalty if the detection class (yellow vs. jute) does not match the track's established class.

The tracker then applies the Hungarian algorithm to this cost matrix, identifying the globally optimal bipartite matching that minimizes the total assignment cost.

Once matched, the internal state of each `Track` is updated. The library utilizes a Kalman Filter to recursively smooth the position and covariance of the tracked object over time; a track is only confirmed as detected after it successfully registers at least two hits across consecutive frames, to avoid spurious detections from being registered as valid. Once a track is confirmed, it is assigned a unique global ID, and its smoothed 3D coordinates are safely passed to the GTSAM optimizer to be added to the map.

4.2 Graph SLAM Integration

The core of the mapping system is a *semantic factor graph* provided by the `orchards-semantic-slam` library, which acts as a structured wrapper around the GTSAM optimization engine [2]. This graph jointly optimizes the robot's trajectory and the positions of the detected plants by representing them as interconnected nodes.

To prevent the factor graph from bloating with redundant data during slow movements, odometry updates are decimated; a new pose node is only added to the graph if the robot has translated by more than a fixed distance or rotated by more than

a fixed angle. When a new odometry node is added, the library generates a factor which mathematically constrains the new pose relative to the previous one, based on the wheel odometry and a configurable covariance noise model.

When a confirmed marker is observed, the system links the robot's historical pose node to a landmark node with a factor, which constrains the graph using the Euclidean distance (range) and the unit vector (bearing) of the relative detection.

To solve the graph, the library uses ISAM2 (Incremental Smoothing and Mapping), which is highly efficient as it updates the graph incrementally as new factors arrive.

4.3 Drift Mitigation via Anchoring

Since the task requires only the distance of the marker from the start of the row, a new graph is initialized every time the robot enters a new row. This also excludes the odometry drift that could be accumulated during the headland turns.

In order to further refine the localization of the markers, two anchor nodes are placed at the start and end of the row. When the robot enters a new row, a *prior factor* is injected into the graph to anchor the starting pose at the local origin. When the robot exits the row, the node injects a final *prior factor* at the end of the trajectory, fixing the final X-coordinate to the known field length. By anchoring both ends of the row and forcing a final GTSAM optimization, the accumulated odometry drift is smoothed and distributed backward across the entire trajectory, resulting in more accurate distance measurements for the mapped plants.

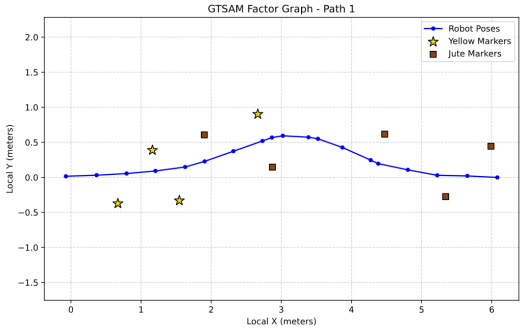


Figure 6: Example visualization of the GTSAM Factor Graph, showing the smoothed robot trajectory (blue) and the optimized positions of the detected yellow and jute markers.

5 Execution and Feedback

5.1 Real-Time Visual Signaling

To fulfill the competition requirement of providing a clear public indication when a diseased plant is detected, the system integrates a custom LED matrix mounted on the robot's chassis.

To prevent transient sensor noise from triggering the visual signal spuriously, the activation is tightly coupled with the multi-object tracker's confirmation logic. Once a plant track reaches the required hit threshold and is officially determined to be a valid detection, an arrow is displayed on the LED matrix, pointing left or right depending on which camera signaled the detection.



Figure 7: Example of Grasslammer with LED matrix attached starting a run.

6 Conclusions

In conclusion, the architecture developed for Task 2 provides a robust and comprehensive solution for autonomous plant health monitoring in unstructured agricultural environments. By combining dynamic HSV color calibration with rigorous 3D geometric and depth validation, the perception layer effectively mitigates the high false-positive rates typically caused by variable outdoor lighting and visual occlusion. Furthermore, the integration of the `orchards-semantic-slam` library bridges the gap between local visual detections and global mapping: by formulating the localization problem as a semantic factor graph and utilizing row-end anchoring, the system mitigates accumulated odometry drift.

While the detection system was robust in regards to false positives, it was sometimes too rigid, or susceptible to changes in the orientation of the

marker, as the calibration process would capture the colour of the marker in a specific lighting and orientation, thus causing some markers to not be detected. To mitigate this a wider HSV range should be considered, for example by combining backprojections over different histograms, or manually inflating the frequencies of a computed histogram with H and S values adjacent to the ones actually extracted.

The Graph SLAM localization system was functional, but failed to achieve the 10cm precision required by the competition during live testing. This could be improved by tuning the parameters of the GTSAM graph through further testing, to ensure that the odometry drift compensation is more accurate.

References

[1] D. Rapado-Rincon and G. Kootstra, "Tree-slam: semantic object slam for efficient mapping of individual trees in orchards," *Smart Agricultural Technology*, p. 101439, 2025.

[2] Wageningen University & Research, "orchards-semantic-slam." `https://github.com/wur-abe/orchards-semantic-slam`, 2025.